

Notice explicative

Algorithme de Map Matching et scripts associés

Table des matières

1	Objet de la notice	1
2	Principe de l'algorithme	2
2.1	Le problème	2
2.2	Modèle de Markov caché (HMM)	2
2.3	Viterbi et <i>backtrace</i>	2
2.4	Chaîne de traitement de bout en bout	2
3	Organisation des scripts	3
4	MapMatching.py en détail	3
4.1	Partie données	3
4.2	Partie géométrie	3
4.3	Partie graphe	3
4.4	Partie stochastique	4
4.5	Viterbi et tracé	4
5	Scripts auxiliaires	4
6	Exécution pas à pas	4
7	Modifications apportées au code d'origine	5
8	Limites connues et pistes d'amélioration	5

1 Objet de la notice

Cette notice accompagne le code de map matching développé dans le cadre du projet de conception (L2S / CentraleSupélec). Elle décrit (i) le principe de l'algorithme, (ii) l'organisation et le fonctionnement des scripts Python, (iii) la procédure d'exécution, et (iv) les modifications apportées par rapport au code d'origine ainsi que les limites connues. Le détail mathématique figure dans le rapport de projet ; cette notice se concentre sur la mise en correspondance entre la théorie et l'implémentation.

2 Principe de l’algorithme

2.1 Le problème

On dispose d’une *trace GPS* (suite de points $z_1, \dots, z_n$ bruités) et d’un *réseau routier* (graphe issu d’OpenStreetMap, composé de nœuds et de segments). On cherche l’itinéraire le plus vraisemblable sur le réseau ayant pu produire cette trace. La projection de chaque point sur la route la plus proche ne suffit pas : elle produit des itinéraires incohérents (demi-tours, oscillations entre routes parallèles). On adopte donc une approche globale et probabiliste.

2.2 Modèle de Markov caché (HMM)

Le réaligement est formulé comme l’inférence des états cachés d’un *modèle de Markov caché* :

- les **observations** sont les points de la trace GPS ;
- les **états cachés** sont les segments de route candidats au voisinage de chaque point ;
- la **probabilité d’émission** mesure la vraisemblance qu’un point GPS provienne d’un segment donné ; elle suit une loi normale de la distance point-segment :

$$p(z \mid r) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}(\|z - x\|_{gc}/\sigma)^2\right);$$

- la **probabilité de transition** pénalise les changements de route peu plausibles ; elle est exponentielle de l’écart d entre la distance à vol d’oiseau et la distance de graphe entre projetés :

$$p(d) = \frac{1}{\beta} \exp(-d/\beta).$$

Les paramètres σ (dispersion du bruit GPS) et β (échelle de l’écart des distances) s’estiment par des médianes robustes (facteurs 1,4826 et $1/\ln 2$) sur des traces de référence.

2.3 Viterbi et *backtrace*

L’algorithme de Viterbi calcule, par programmation dynamique, le chemin de probabilité maximale dans la maille « observations $\times$ états ». À chaque étape i et pour chaque état s_l , on conserve la meilleure probabilité d’atteindre s_l et l’état précédent s_m qui la réalise :

$$p(e_1 \dots e_i, s_l) = \max_{s_k \in S} (p(e_1 \dots e_{i-1}, s_k) p(s_k; s_l) p(s_l; e_i)).$$

Une fois la dernière étape atteinte, la phase de *backtrace* remonte les pointeurs s_m pour reconstituer la séquence de segments, projetée pour obtenir l’itinéraire final.

2.4 Chaîne de traitement de bout en bout

1. **Lecture des données** OpenStreetMap (JSON) : séparation des nœuds et des routes.
2. **Construction des segments** et densification du maillage par des *nœuds fictifs* régulièrement espacés (améliore la détection de proximité).
3. **Sélection des candidats** : pour chaque point GPS, segments situés dans un rayon donné.
4. **Construction du graphe** (nœuds réels + projetés) pour les calculs de plus court chemin (Dijkstra).
5. **Matrices d’émission B et de transition A** (normalisées par ligne), recalculées le long de la trace.
6. **Viterbi + backtrace** : séquence de segments la plus probable.
7. **Tracé** : carte, trace GPS, itinéraire reconstitué.

3 Organisation des scripts

Fichier	Rôle
MapMatching.py	Script principal : chaîne complète (données, géométrie, graphe, partie stochastique, Viterbi, tracé).
code_thomas.py	Version préliminaire de <code>viterbi/run</code> , conservée à titre historique.
MapMatching_test_5.py	Prototype de visualisation sur données synthétiques (coordonnées cartésiennes jouets, petit extrait <code>small.json</code>).
*.gpx	Traces GPS d'exemple (secteurs Corbeville, Guichet).
README.txt	Mode d'emploi succinct.

Table 1 – Fichiers du projet.

Dépendances : Python 3, numpy, matplotlib (pip install numpy matplotlib).

4 MapMatching.py en détail

Le script s'organise en cinq parties.

4.1 Partie données

transfo_coord(lat, lon)

convertit des coordonnées géographiques en coordonnées planes (projection vers un repère cartésien centré sur la Terre).

list_node(json) / segments_node(json)

extraient respectivement la liste des nœuds et la liste des *segments* (couples de nœuds consécutifs d'une même route). Un test de redondance évite de compter deux fois le même segment.

list_points(json, seg_list)

renvoie un dictionnaire {identifiant de nœud → [coordonnées, segments incidents]}. C'est ici que sont insérés les **nœuds fictifs** (densification du maillage, pas réglé par la variable `m`).

trackpoint_creation(trackpoint, json)

transforme la liste de couples [lon, lat] en structures de nœuds exploitables.

adjacent_seg(...) / **ensemble_segments(...)**

sélectionnent les segments candidats au voisinage d'un point (rayon fixé par la fonction `voisins`, ici ≈ 50).

4.2 Partie géométrie

`dis_pt_pt`, `vecteur`, `produit_scalaire`, `pt_projete` et `dist_pt_segments` calculent distances, projections orthogonales et distance point-segment. `dist_pt_segments` distingue, via des produits scalaires, le cas où le projeté tombe sur le segment de celui où il tombe au-delà d'une extrémité (on retient alors le nœud le plus proche).

4.3 Partie graphe

graphe(json, traces)

construit le graphe pondéré (listes d'adjacence) à partir des segments, et y insère les projetés des traces.

dijkstra(pt1, G)

calcule les plus courtes distances depuis **pt1** (renvoie un dictionnaire {noeud → {parent, distance}}).

chemin(pt1, pt2, G)

reconstruit le plus court chemin par remontée des parents.

4.4 Partie stochastique

normalisation(liste)

normalise une ligne de probabilités (somme unité).

matrice_emission(...)

remplit B : pour chaque couple (segment candidat, trace), la probabilité d'émission, puis normalise par ligne.

matrice_transition(t, ...)

remplit A à l'instant t : probabilité de transition entre segments candidats des étapes t et $t + 1$, à partir de la distance de graphe (Dijkstra) comparée à la distance à vol d'oiseau.

compute_sigma / compute_beta

estiment σ et β (*non branchées sur la démonstration — voir §8*).

4.5 Viterbi et tracé

viterbi(...) initialise l'état de départ, construit B et A , applique la récurrence de Viterbi puis **backtrace(...)** pour renvoyer la séquence de segments. **plot_routes(...)** affiche la carte (noir), la trace GPS (rouge) et le matching (bleu).

5 Scripts auxiliaires

`code_thomas.py` contient une version antérieure de **viterbi/run** (elle s'appuie sur les fonctions de `MapMatching.py`, à importer). Elle est conservée comme trace du travail de conception.

`MapMatching_test_5.py` est un banc d'essai sur données synthétiques 2D : il a servi à mettre au point l'affichage des routes, la projection des points et la détection des intersections, avant le passage aux coordonnées géographiques réelles.

6 Exécution pas à pas

1. Renseigner `JSON_PATH` (en tête de `MapMatching.py`) avec le chemin du fichier OpenStreet-Map, et compléter la liste `trackpoint` avec les traces à réaligner.
2. **Phase 1** — décommenter, dans le bloc `__main__`, le calcul de G , D , V et `print(V)` ; exécuter ; récupérer la liste de segments affichée.
3. **Phase 2** — coller cette liste dans `seq`, décommenter l'appel à `plot_routes`, exécuter ; la figure du matching s'affiche.

```
# Phase 1 : génération de la séquence de segments
G = graphe(data_dict, traces)
D = dist_pt_segments(traces, data_dict)
V = viterbi(traces, liste_segments, 2, 2, data_dict, G, D, seg_list)
print(V)

# Phase 2 : tracé (seq = liste obtenue en phase 1)
seq = [...]
plot_routes(data_dict, traces, seq, seg_list, liste_segments)
```

7 Modifications apportées au code d'origine

Le code a été porté en Python 3 et rendu exécutable ; la logique a été préservée. Chaque correction est signalée par un commentaire [FIX] dans les sources.

Fichier	Correction
MapMatching.py	Imports explicites (<code>sqrt</code> , <code>floor</code> , <code>pi</code> , <code>exp</code>) ; <code>zeros</code> → <code>np.zeros</code> ; <code>np.int</code> → <code>int</code> ; chemin de fichier rendu configurable (<code>JSON_PATH</code>) ; bloc d'essai protégé par <code>__main__</code> .
MapMatching.py (Viterbi)	Correction d'un décalage d'indice : la boucle parcourt désormais $t = 1, \dots, T - 1$ (l'originale s'arrêtait en $T - 2$, laissant la dernière colonne non calculée) et utilise la transition de $t - 1$ vers t , conformément à la version de référence laissée en commentaire par les auteurs. Valeur par défaut pour <code>q_init</code> .
code_thomas.py	Ajout de <code>import numpy as np</code> ; <code>len(T)</code> → <code>T</code> ; <code>np.int</code> → <code>int</code> .
MapMatching_test_5.py	Chemin configurable ; <code>rdm.sample</code> sur <code>list(...)</code> ; correction de <code>.append=</code> → <code>.append(</code> dans <code>make_graphe</code> .

Table 2 – Synthèse des corrections.

8 Limites connues et pistes d'amélioration

Complexité.

La construction de `matrice_transition` relance un Dijkstra pour chaque couple de segments candidats, à chaque pas de temps. Le coût croît rapidement avec la taille du graphe et le nombre de candidats, ce qui limite l'usage à de petits secteurs (cf. rapport, §5). Pistes : mise en cache des plus courts chemins, restriction du nombre de candidats par point, structures spatiales (grille / *k-d tree*) pour la recherche de voisinage.

Formule d'émission.

L'implémentation calcule `exp(-0.5*(d/sigma))` (sans élévation au carré), alors que la formule du rapport est une gaussienne en $(d/\sigma)^2$. À aligner si l'on souhaite respecter strictement le modèle.

Estimation des paramètres.

`compute_sigma` et `compute_beta` nécessitent des traces de référence (vraie route connue) et présentent des incohérences de signature (`pt_proje` attend quatre arguments ; `dijkstra` renvoie un dictionnaire). Elles ne sont pas branchées sur la démonstration, qui fixe $\sigma = \beta = 2$. À reprendre avant tout usage réel.

Initialisation.

Le choix de l'état initial `q_init` pourrait être affiné (probabilité d'émission du premier point plutôt que simple proximité).

Conversion de coordonnées.

La formule du rapport et celle du code diffèrent légèrement (choix des fonctions trigonométriques) ; sans incidence sur les tests à petite échelle, mais à homogénéiser.